thws Technische Hochschule Würzburg-Schweinfurt

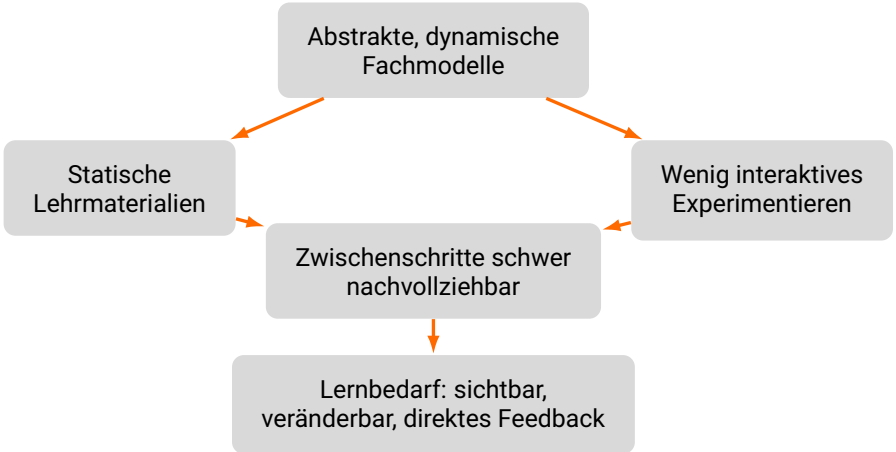
Projektarbeit

INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK

M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

1. KONTEXT UND ZIELBILD

PROBLEMSTELLUNG



thws
Technische Hochschule
Würzburg-Schweinfurt

ZIELE DER PLATTFORM

- Browserbasierte Lern- und Übungsplattform
- Überblick, Beispiele, Übungen und freie Simulation
- Einheitliche Bedienung, Deutsch/Englisch und Hell/Dunkel

2. ANFORDERUNGEN UND PROJEKTABLAUF

ANFORDERUNGEN

- Didaktik
- Bedienung
- Technik
- Qualität

ZUSAMMENARBEIT UND FEEDBACK-ZYKLEN

- Agil orientierte, iterative Arbeitsweise
- Wöchentliches Teammeeting
- GitLab, Work Items und Milestones
- Regelmäßiges Auftraggeber-Feedback

3. LÖSUNGSANSATZ: ARCHITEKTUR UND DESIGN

TECHNISCHE BASIS

- React 19 + TypeScript + React Router
- Vite + Tailwind CSS 4 + shadcn/ui
- Zustand + i18next
- Monaco + OpenJDK javac + TeaVM

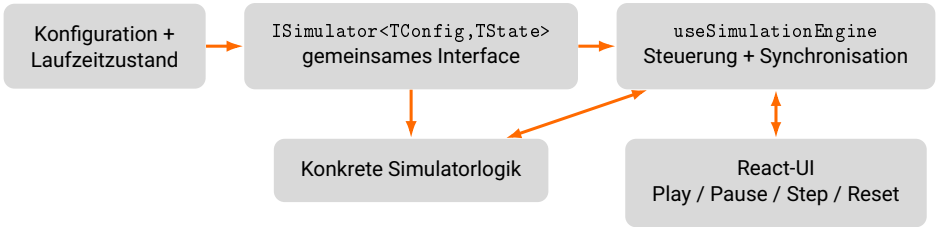
MODULARE PLATTFORMARCHITEKTUR

- Fachlich getrennte Topics
- Gemeinsame Shared Components
- `moduleCatalog.tsx` zur Modulverwaltung

UNIFIED UI: WIEDERKEHRENDES DESIGN

- Header, Sidebar und Content
- Überblick, Übungen, Beispiele und Simulation
- Gemeinsame Komponenten
- Fachspezifische Visualisierung in jedem Topic

SIMULATOR-ARCHITEKTUR



IMPORT, EXPORT UND LOKALE PERSISTENZ

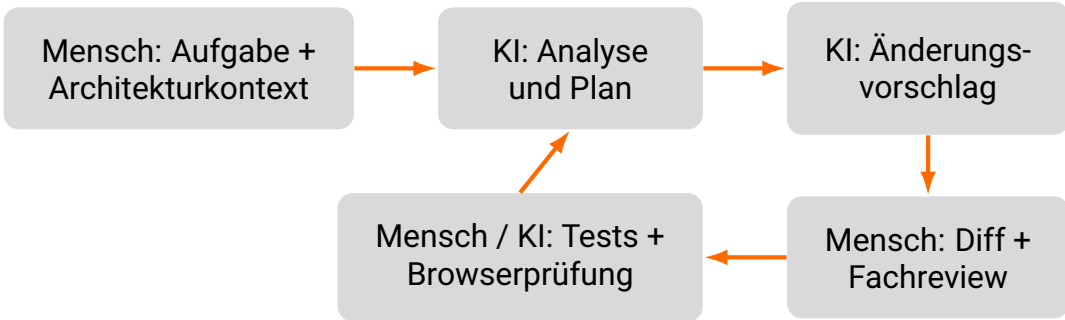
- Gemeinsamer Transfer-Codec
- Kompakte Base64URL-Transferstrings
- Local Storage

4. KI-EINSATZ IM PROJEKT

KI-WERKZEUGE IM PROJEKT

- Stitch, ChatGPT, Gemini, Claude, Codex, Antigravity und lokale Modelle
- MCP, Graphify und Agent Skills

ARBEITSWEISE: KI ALS CO-DEVELOPER



GRENZEN UND ERKENNTNISSE BEIM KI-EINSATZ

- Kontextverlust bei großen Aufgaben → `handover.md`
- Duplikate und toter Code → Code Review mit KI
- Nutzen durch kleine, testbare Schritte

5. QUALITÄT UND DEPLOYMENT-STRATEGIE

QUALITÄTSSICHERUNG

- ESLint + Prettier
- Vitest
- Vite Build + Playwright
- Lighthouse + manuelles Review

DEPLOYMENT

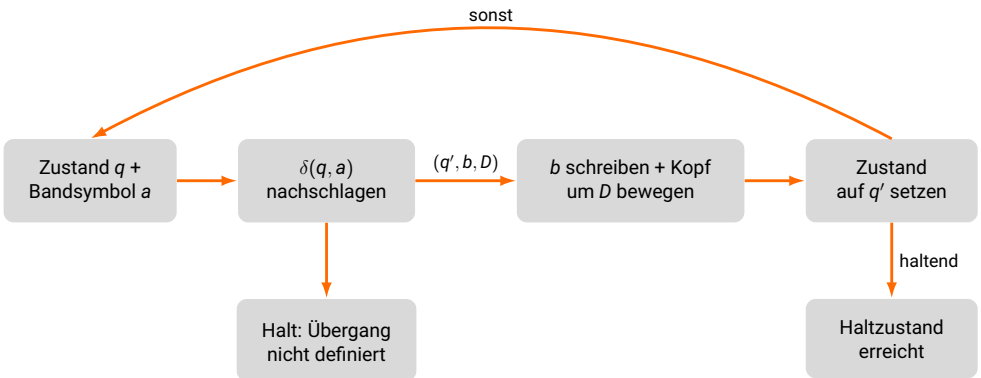
- Statischer Vite-Build
- Browserbetrieb durch lokalen Server
- Online hosting durch Auftraggeber abgelehnt

FACHMODUL: TURINGMASCHINE

TURINGMASCHINE KONFIGURIEREN

- Übergänge als (q', b, D) in der Übergangstabelle
- Zustände, Haltezustände und Anfangsband definierbar
- Eingabe- und Bandalphabet werden automatisch abgeleitet
- Partielle Übergangsfunktionen: Leere Zellen führen zum Halt

SCHRITTAUSFÜHRUNG

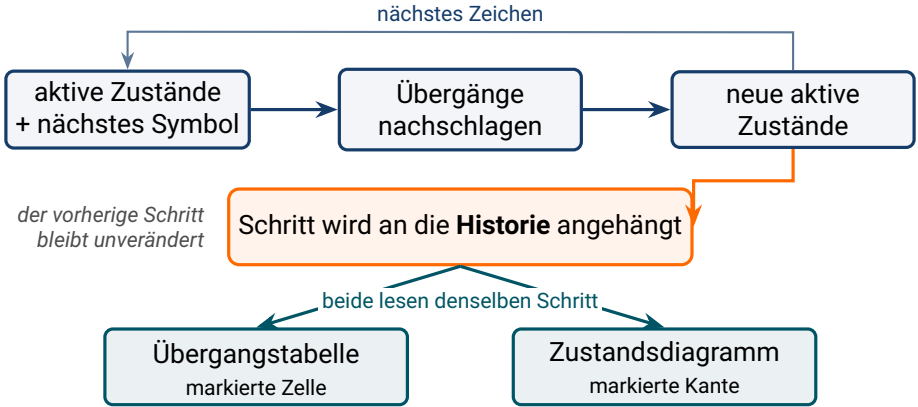


FACHMODUL: ENDLICHE AUTOMATEN

ENDLICHE AUTOMATEN – ÜBERBLICK

- Studierende definieren den Automaten selbst: **Typ (DFA/NFA)**, Alphabet, Start- und Endzustände, Übergänge und Testwort.
- Alternativ entsteht der Automat aus einer **rechtslinearen Grammatik**.
- Der Ablauf ist nicht nur in der **Übergangstabelle** zu sehen, sondern gleichzeitig im **Zustandsdiagramm** – der Weg des Wortes bleibt leicht zu verfolgen.
- **Stapeltest**: viele Wörter auf einmal prüfen – erkennt der Automat die beabsichtigte Sprache?
- **Schritt-Historie**: vorwärts, rückwärts und zurück zu einem früheren Schritt.
- Übungen geben eine automatische Rückmeldung mit **Gegenbeispiel**.

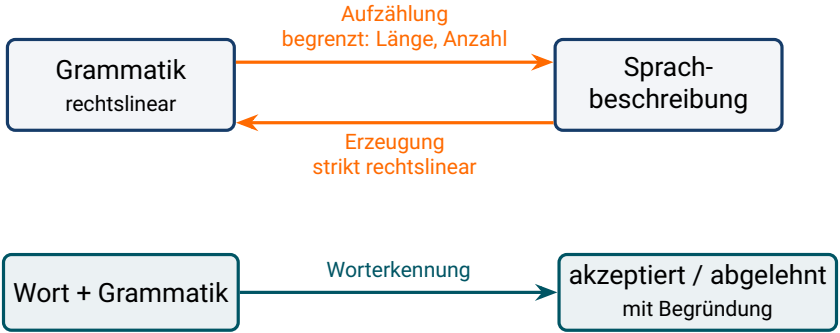
EIN SIMULATIONSSCHRITT – UNTER DER HAUBE



DFA: ein aktiver Zustand, **NFA:** mehrere – derselbe Ablauf, kein `if (typ === 'DFA')`.
Der Typ steuert nur die **Validierung**: doppelte Übergänge sind beim DFA ein Fehler.

FACHMODUL: REGULÄRE SPRACHEN

REGULÄRE SPRACHEN – DREI ARBEITSWEISEN



Alle drei Modi bleiben im Hintergrund erhalten und werden nur ein- oder ausgeblendet – dadurch überlebt eine begonnene Eingabe den Moduswechsel.

BEWUSST ABGELEHNTE SPRACHBESCHREIBUNGEN

```
L = {a^n b^n | n > 0}
```

nicht regulär – dieselbe Variable in zwei Faktoren koppelt die Wortlängen aneinander

```
L = {a^n | n < 5}
```

obere Schranke – beschreibt zwar eine endliche Sprache, ist als Aufzählung aber klarer

Beide Eingaben ließen sich formal verarbeiten. Sie werden trotzdem mit einer **eigenen Meldung** zurückgewiesen: Diese Fälle ausdrücklich zu erklären ist lehrreicher, als sie stillschweigend zu verwerfen.

ÜBUNGSPRÜFUNG: VERHALTEN STATT STRUKTUR

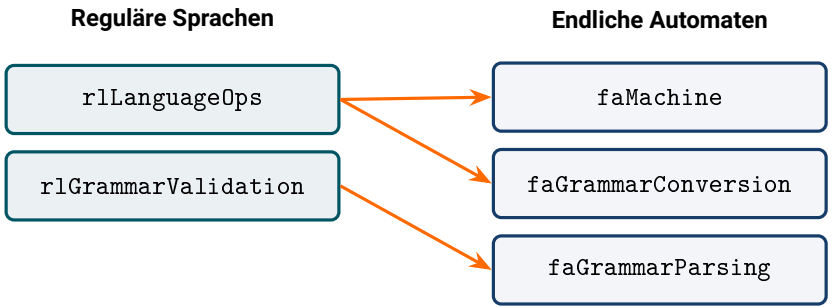


gleiche Entscheidung für jedes Wort?

ja ⇒ bestanden
nein ⇒ erstes abweichendes Wort als Gegenbeispiel

Ein **Strukturvergleich** würde korrekte Lösungen ablehnen, die nur anders aufgebaut sind. Die begrenzte Wortmenge ist bewusst eine **didaktische Rückmeldung** und kein formaler Äquivalenznachweis. Strukturelle Mängel werden vorrangig gemeldet – sie sind meist die eigentliche Ursache.

WIEDERVERWENDUNG STATT DUPLIKAT



Abhängigkeit nur in eine Richtung – kein Zyklus

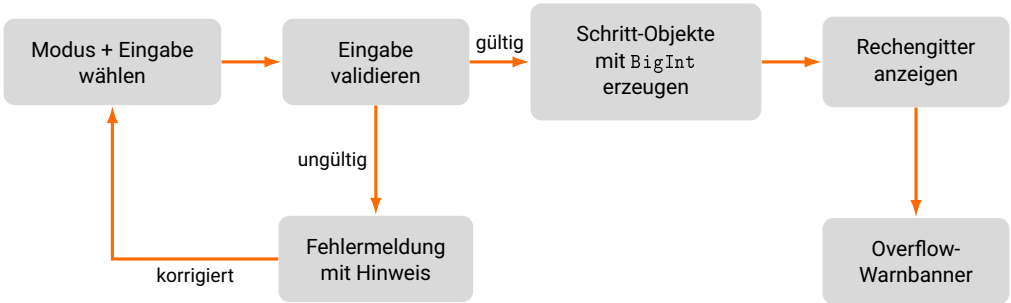
Das Verzeichnis `domain` hat **keine Abhängigkeit zu React** und bleibt ohne gerenderte Oberfläche testbar. Die Aufzählung führt eine Grammatik über denselben Automatenkern aus wie die Simulation – eine eigene Ableitungslogik hätte zwei Implementierungen desselben Verhaltens erfordert.

FACHMODUL: ZAHLENSYSTEME

ZAHLENSYSTEME KONFIGURIEREN

- Fünf Modi: *Integers*, *Negative*, *Fractional*, *IEEE-754* und *Arithmetik* – Übung oder Beispiel ordnet sich automatisch dem passenden Modus zu
- Eingaben je nach Basis mit Regex-Filtern validiert, Bitlänge explizit begrenzt
- Negative Zahlen in fester 8-Bit-Breite (Wertebereich −128 bis 127, abgestimmt auf die Vorlesung)
- Berechnungen durchgängig mit `BigInt` statt `Number`: keine Rundungsfehler bei großen Zahlen
- 10 Übungen und 36 vorlesungsbegleitende Beispiele mit automatischer Rückmeldung

SCHRITTAUSFÜHRUNG

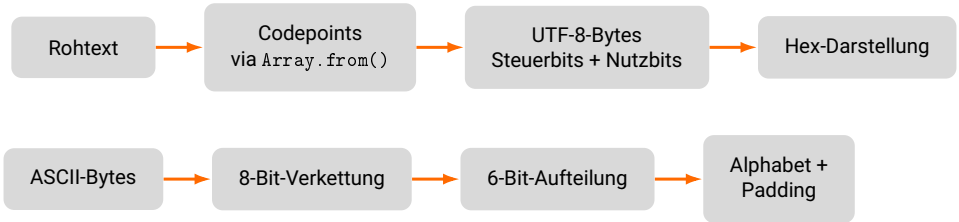


FACHMODUL: KODIERUNG

KODIERUNG KONFIGURIEREN

- Zwei Verfahren vollständig umgesetzt: **UTF-8** und **Base64**
- Freier Text oder Übungsaufgabe als Eingabe – Ver- und Entschlüsselung unterstützt
- Strenge Decodierung: *Overlong Encodings* werden als Sicherheitsrisiko abgelehnt, nicht druckbare Zeichen erscheinen als Escape-Sequenzen
- Übungen dynamisch generiert (easy = ASCII, medium = 2-Byte, hard = 3-Byte-Unicode) – Fortschritt dabei bewusst nicht persistent
- Zustand nur im freien Simulator automatisch gespeichert

TRANSFORMATIONSSCHRITTE



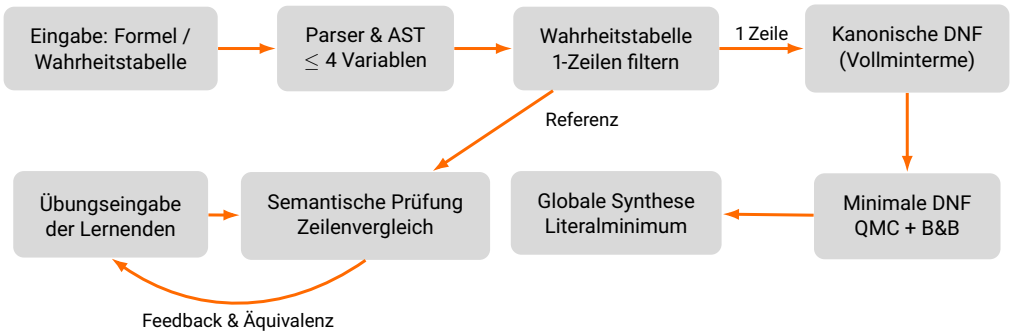
Jeder Pfeil steht für eine eigene aufklappbare Ansicht – Studierende entscheiden selbst, wie tief sie einsteigen. Bewusst keine Animationen: die Schritte bleiben statisch und im eigenen Tempo nachvollziehbar.

FACHMODUL: DISJUNKTIVE NORMALFORM

DISJUNKTIVE NORMALFORM – ÜBERBLICK

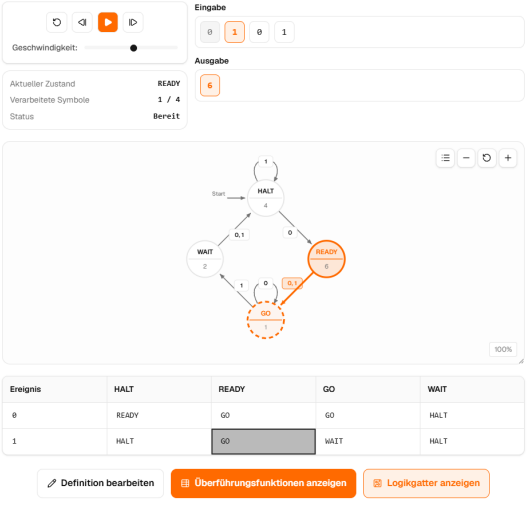
- Zwei Aufgabenmodi: Wahrheitstabelle nach DNF und umgekehrt
- Drei Stufen: kanonische DNF, minimale DNF und Minimalform
- Minimierung mit Quine-McCluskey und Branch-and-Bound
- Semantische Prüfung erkennt logische Äquivalenz
- Begrenzung auf maximal 4 Variablen für exakte Synthese

SCHRITTAUSFÜHRUNG



FACHMODUL: MOORE MEALY

INTERAKTIVE SIMULATOR DARSTELLUNG



- Automaten frei konfigurieren
- Eingaben Schritt für Schritt simulierbar
- Aktueller Zustand und Übergang direkt sichtbar
- Vor- und Zurückspringen in der Simulation

AUTOMAT ANALYSIEREN & SCHALTUNG DARSTELLEN

Überführungsfunktion

- Zustandsübergänge übersichtlich darstellen
- Ausgabe abhängig von Moore bzw. Mealy

Wahrheitstabelle

- Zustände und Eingaben binär codieren
- Boolesche Funktionen automatisch ableiten
- DNF minimieren

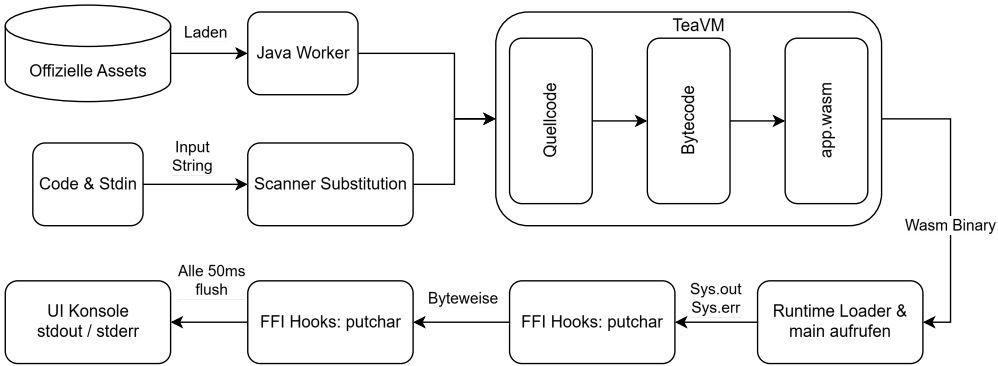
Logikgatter

- Funktionen als Schaltung visualisieren
- Gatterdarstellung
- Verbindung von Automat und Hardware

→ **logische bzw. hardwareseitige Umsetzung sichtbar machen**

FACHMODUL: JAVA

JAVA – ÜBERBLICK



BESONDERHEITEN UND EINSCHRÄNKUNGEN

- **Offline nutzbar** – Verteilung als ZIP, keine externe Verbindung nötig
- **Kein lokales Java** – kein JDK, Gradle oder Git erforderlich
- **Kein Java-Backend** – Kompilierung und Ausführung lokal im Browser
- **Plattformunabhängig** – Nutzung auch auf Tablets / iPads im Browser
- **Browser-Sandbox** – kein interaktives `System.in`, kein Dateisystemzugriff
- **Für Lernaufgaben ausreichend** – Code, (Eingaben) und `stdout`/`stderr`

6. INTERAKTIVE LIVE-DEMO

DEMO-ZIEL: DEN LERNWEG SICHTBAR MACHEN

- Eingabe verändern
- Schrittweise ausführen
- Zwischenschritte erklären

7. AUSBLICK UND FAZIT

PROJEKTERGEBNISSE

- Modulare Plattformarchitektur
- Gemeinsame Simulatorbasis und Transfer
- Automatisierte Qualitätssicherung

PERSPEKTIVEN UND OFFENE PUNKTE

- Weitere Lernbereiche und Topics
- mobile Nutzung verbessern
- LMS-Anbindung